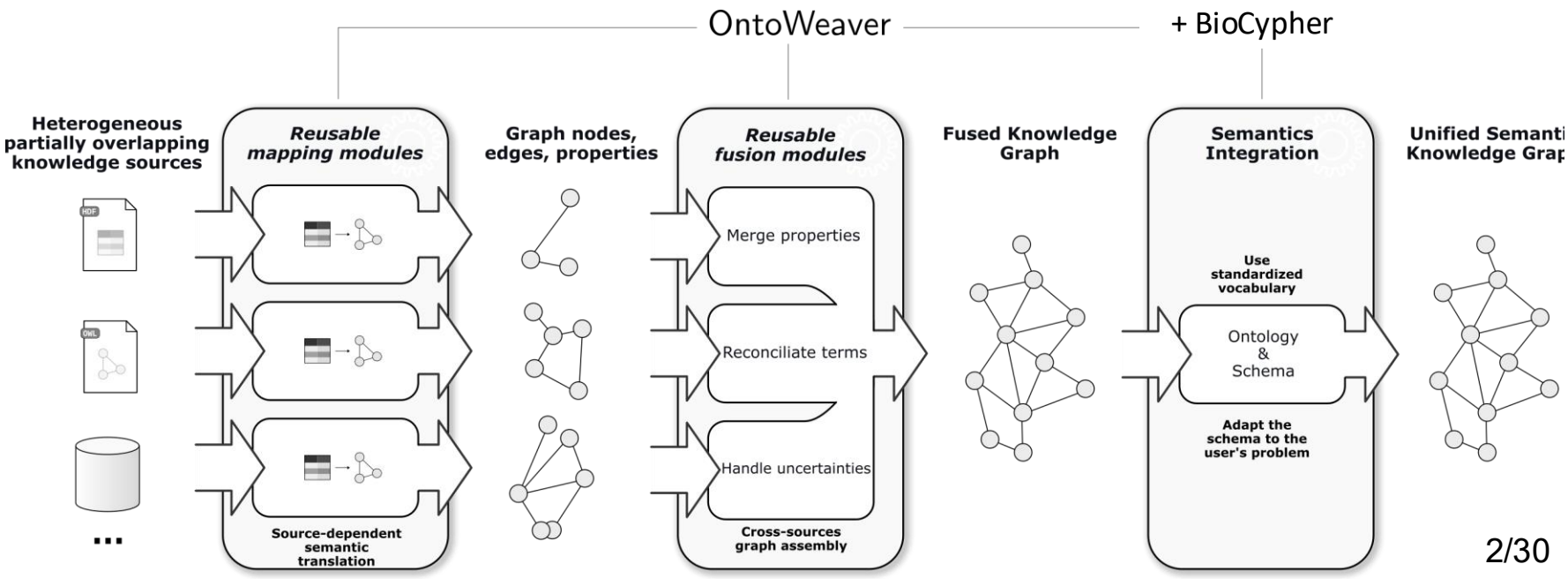
Hands on Fusion

with OntoWeaver

Johann Dreo, Claire Laudy, Matthieu Najm, Benno Schwikowski
Computational Systems Biomedicine Lab
Institut Pasteur

Biomedical Knowledge Graph Integration is Hard



Advanced OntoWeaver: fusion-enabled combination

```
$ ontoweave
    | data_A1.csv:mapping_A.yaml
    | data_A2.csv:mapping_A.yaml
    | data_B/*.parquet:mapping_B.yaml
    | ontology.ttl:automap
```

Simplest sane defaults:

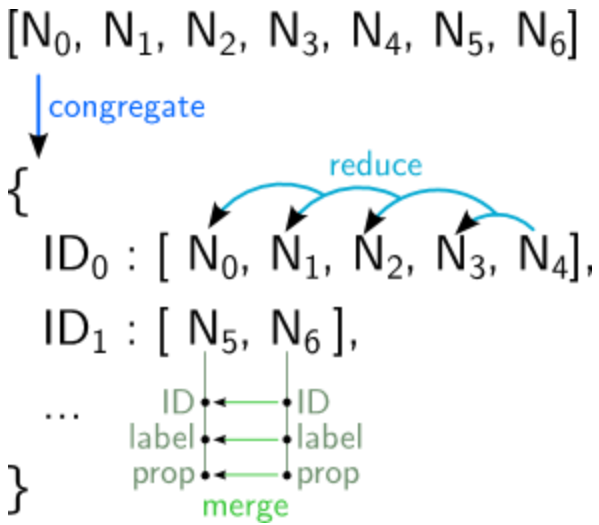
- keep all property values
  - (can be anything else)
- align on ID+Label
  - (can be any serialization)

Default fusion in ontoweave

```
# Call the mappings:
adapter_A = ontoweaver.tabular.extract_table(input_table_A, mapping_A)
adapter_B = ontoweaver.tabular.extract_table(input_table_B, mapping_B)

# Aggregate the nodes and edges:
nodes = adapter_A.nodes + adapter_B.nodes
edges = adapter_A.edges + adapter_B.edges

# Reconciliate:
fused_nodes, fused_edges = ontoweaver.fusion.reconciliate(nodes, edges, reconciliate_sep=";")

# Then you can pass those to biocypher.write_nodes and biocypher.write_edges...
```

ontoweaver.fusion.reconciliate

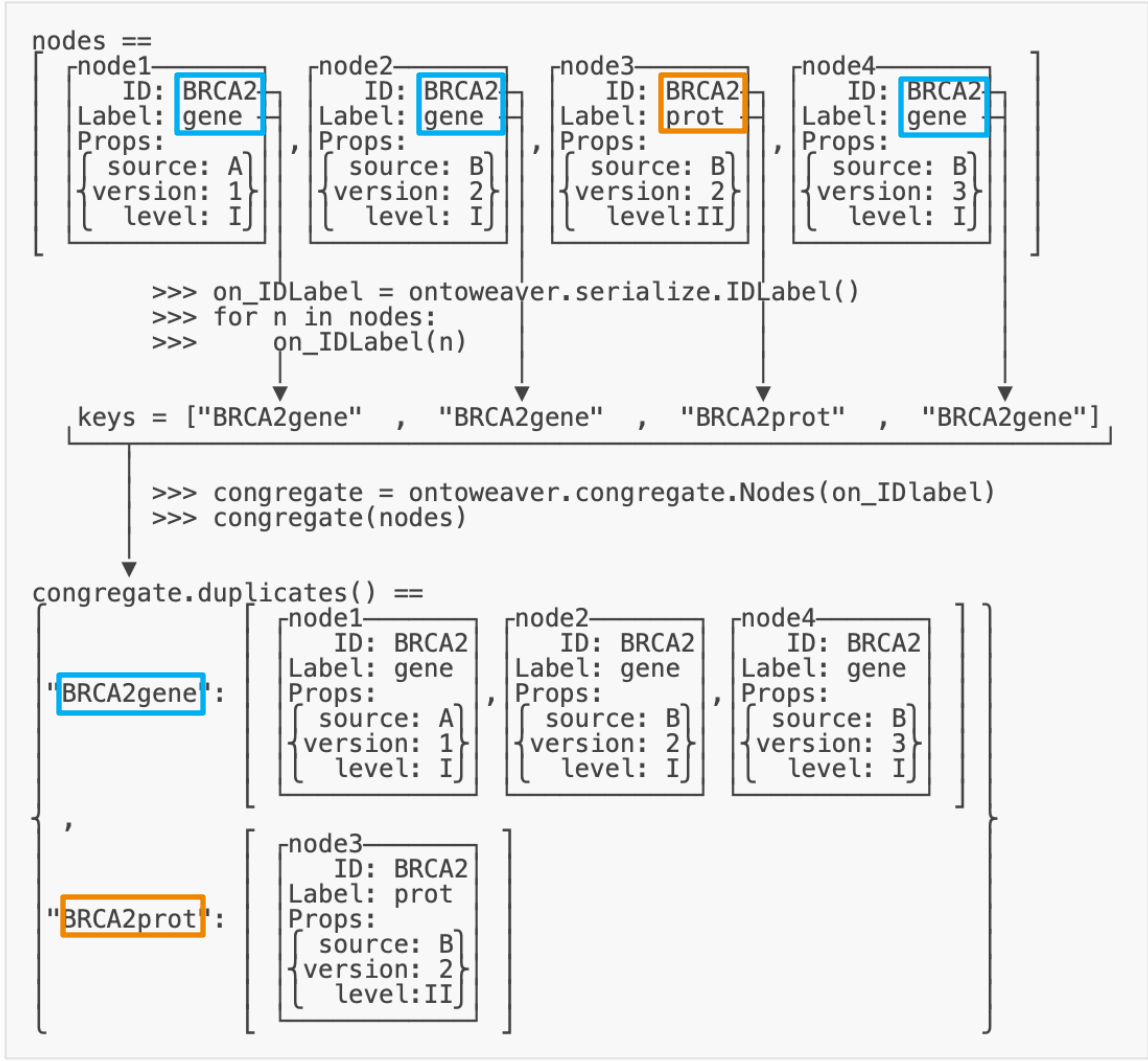
- Congregate:
  - Serialize: ID + Type Label
  - Allows $\mathcal{O}(n \log n)$
- Reduce:
  - Merge pairs of nodes
  - Properties by properties
- Merge:
  - Aggregate property values
  - In a string-list



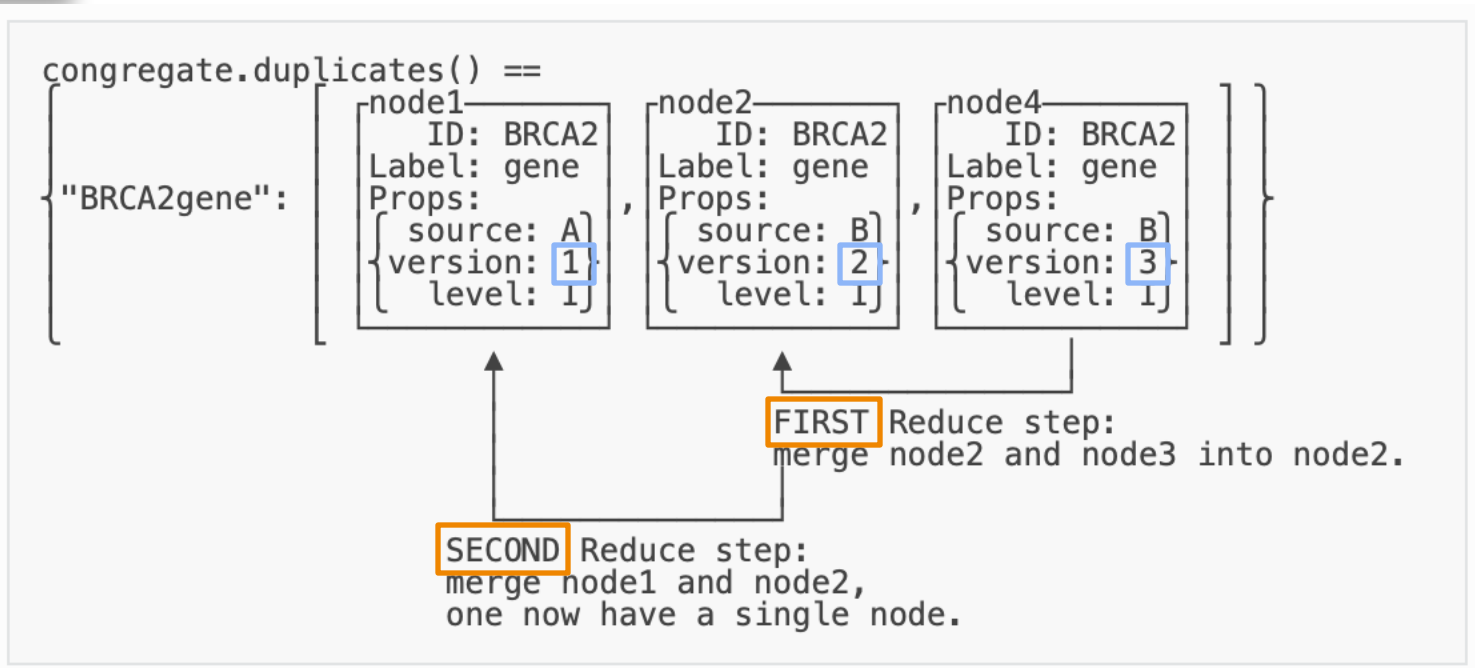
Congregate

$[N_0, N_1, N_2, N_3, N_4, N_5, N_6]$

congregate



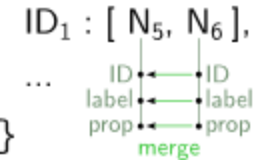
Reduce


congregate.duplicates() ==
"BRCA2gene":
node1
ID: BRCA2
Label: gene
Props:
source: A
version: 1
level: 1
node2
ID: BRCA2
Label: gene
Props:
source: B
version: 2
level: 1
node4
ID: BRCA2
Label: gene
Props:
source: B
version: 3
level: 1
FIRST Reduce step:
merge node2 and node3 into node2.
SECOND Reduce step:
merge node1 and node2,
one now have a single node.


reduce
ID0 : [N0, N1, N2, N3, N4],

Merge

```
fuse.Members.merge \
(
  key
  BRCA2gene ,
  node1
     ID: BRCA2
  Label: gene
  Props:
  { source: A  }
  {version: 1  }
  {  level: I  }
,
  node2
     ID: BRCA2
  Label: gene
  Props:
  { source: B  }
  {version: 2,3}
  {  level: I  }
)
  ------UseFirst------>
  --EnsureIdenticals-->
  ------Append------>
  --------{A}+{B}-------->
  --------{1}+{2,3}----->
  --------{I}+{I}-------->
  node
     ID: BRCA2
  Label: gene
  Props:
  { source: A,B   }
  {version: 1,2,3}
  {  level: I     }
```



Look at the code

ontoweaver/src/ontoweaver/fusion.py:132

def **reconciliate_nodes**(...)

Note on the design pattern

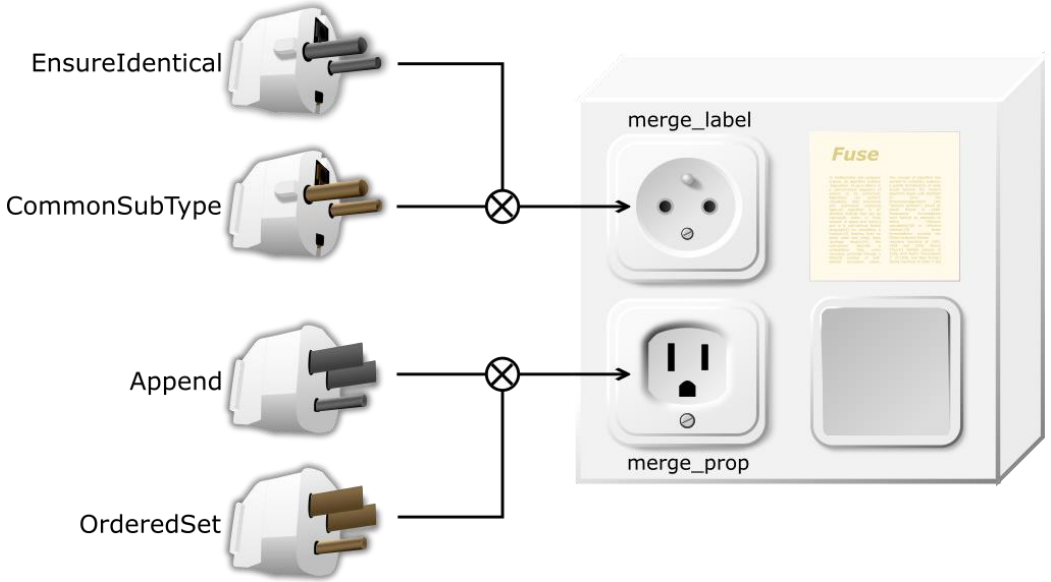
- Object-oriented programming
- Composition of functions
  - That carry a state
  - Sometimes called “closures” in functional programming
- Design pattern: *Strategy* + *Functors*

```
instance = module.Class(state)
result = instance(input) # call
    # Equivalent to:  result = instance.__call__(input)
```

```
op1 = module.Op1(state1)
op2 = module.Op2(state2)
op = module.Class(op1, op2, parameter)
result = op(input)
```

Compose operators

```
idt = EnsureIdentical()
lst = Append(“;”)
fuser = Members(Node, idt, lst)
result = fuse(nodes)
```



Learn more at:
https://github.com/nojhan/algopattern

Fusion architecture

- **Three levels of abstraction:**

  1) merge.Merger —(used by)→ fuse.Members —(used by)→ fusion.***Reduce***
  2) fuse.Fuser —(used by)→ fusion.***Reduce***
  3) fusion.***Fusioner***

- **Why?**

  1) Merger : pairs of *atomic variables* (strings, dict, ...)
  2) Fuser : *pairs* of *elements* (nodes or edges)
     - e.g. climb the type hierarchy to reach a more generic type
  3) Fusioner : *lists* of elements
     - e.g. you need to know all duplicated nodes before deciding which type to set

Edge remapping

- What if one changes the nodes' IDs during fusion?
    - We would have undefined edges' tips
- We then need to *remap* changed node IDs within edges
- That's why `reconciliate_node` returns an `ID_mapping`
- Which you can then use with `remap_edges`
- ***Before*** edges fusion!

fusion.py:275
def reconciliate(...)

Example of mergers

```
class UseFirst(StringMerger):
    def merge(self, key, lhs: str, rhs: str) -> str:
        self.set(lhs)

class EnsureIdentical(StringMerger):
    def merge(self, key, lhs: str, rhs: str) -> str:
        if self.merged:
            if self.merged != lhs or self.merged != rhs:
                raise ValueError(f"Merged {lhs}/{rhs} ≠ {self.merged}")
            else:
                if lhs != rhs:
                    raise ValueError(f"Merged {lhs} ≠ {rhs} ")
            self.set(lhs) # Should be equal to rhs.
```

ontoweaver/src/ontoweaver/merge.py
ontoweaver/doc_built/_autosummary/ontoweaver.merge.html#ontoweaver.merge.Merger

Try it

cd ontoweaver ; git pull

Modify tests/test_fusion.py, to:

1. Merge IDs using the first encountered
2. Merge labels using the last encountered
3. Merge properties using the **higher ESCAT tiers**

```
if=$(uv run tests/test_fusion.py) && tail $(dirname $if)/biocypher.ttl
```

Fusion module perspectives

- Current state of the fusion module is less user-friendly than mapping: config files?
- Implement more fusion operators
  - Need use cases ☺

- Integrate fusion as a BioCypher deduplication? Or keep it separated from OntoWeaver?
  - Would allow more flexible workflows?
- Implement a declarative approach?
- Implementation(s) for cases that do not fit in-memory

Thanks

Marko Baric, Taru Muranen, Altti Ilari Maarala, Sebastian Lobentanzer, Edwin Carreño, Jaana Oikkonen, Federico Bolelli, Vittorio Pipoli, Veli-Matti Isoviita, Ekaterina Gaydukova, Emanuele Quaglio, Inga Ulusoy, Johanna Hynninen

Codes: https://github.com/oncodash

Corresponding author: johann.dreo@pasteur.fr

Looking for:

- **use cases (for our main publication)**
- issues
- pull requests
- collaborations
- funding project ideas

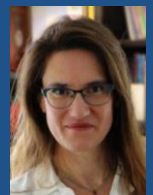
DECIDER

Improving clinical decisions in cancer

OntoWeaver

OncodashKB

Fusion



Fusion architecture (UML)